

MODULE 8 – Modules & Packages (DEEP DIVE)

From single-file scripts → real Python projects

Why This Module Is IMPORTANT

Till now, your code mostly lived in one file.

Real software:

- Has multiple files
- Each file has a clear responsibility
- Logic is reused, not copied

👉 Modules & Packages = Project structure

Key Terms (Understand First)

Term	Meaning
Module	A single <code>.py</code> file
Package	A folder containing modules
Import	Using code from another module

Namespace **Scope of names in a module**

Standard library **Built-in Python modules**

Keep these words in mind — we'll use them a lot.

8.1 What Is a Module?

👉 Any **.py** file is a module

```
math_utils.py
```

```
python
```

```
def add(a, b):  
    return a + b
```

This file itself is a **module**.

◆ Using a Module (`import`)

python

```
import math_utils
```

```
print(math_utils.add(10, 20))
```

💡 Access functions using:

```
module_name.function_name
```

🟦 8.2 Different Ways to Import (VERY IMPORTANT)

1 Normal Import

python

```
import math_utils
```

- ✓ Safe
- ✓ Clear
- ✗ Slightly longer syntax

2 Import with Alias

python

```
import math_utils as mu  
  
print(mu.add(5, 3))
```

- ✓ Cleaner
- ✓ Very common in real projects

3 Import Specific Functions

python

```
from math_utils import add, subtract  
  
print(add(10, 5))
```

- ✓ Short
- ✗ Can cause name conflicts

4 Import Everything (❌ Avoid)

python

```
from math_utils import *
```

- ❌ Bad practice
- ❌ Confusing
- ❌ Interview red flag

8.3 Why `__name__ == "__main__"` Exists (CRITICAL)

This is a must-know concept.

```
# calc.py
```

```
def add(a, b):  
    return a + b
```

```
print("Calculator Loaded")
```

If imported:

```
python
```

```
import calc
```

Output:

```
nginx
```

```
Calculator Loaded
```



✗ Not desired behavior

Correct Way

```
python

def add(a, b):
    return a + b

if __name__ == "__main__":
    print("Calculator Loaded")
```

Now:

- Runs only when file is executed directly
- Does NOT run when imported



This separates reusable code from executable code



8.4 Python Standard Library (POWER)

Python comes with batteries included.

Common Built-in Modules

Module	Use
math	Mathematical operations
random	Random numbers
datetime	Date & time
os	OS interaction
sys	Python runtime info

math Example

```
python

import math

print(math.sqrt(16))
print(math.factorial(5))
```

random Example

```
python

import random

print(random.randint(1, 6))
```

datetime Example

```
python

from datetime import datetime

now = datetime.now()
print(now)
```


8.5 What Is a Package?

👉 A package is a folder that contains modules.

Example structure:

```
project/  
|  
├─ utils/  
|   ├─ math_utils.py  
|   └─ string_utils.py  
└─ main.py
```

8.6 Creating Your Own Package

Step 1: Folder Structure

```
markdown
```

```
utils/
```

```
    __init__.py
```

```
    math_utils.py
```

`__init__.py` tells Python:
"This folder is a package"

Step 2: Use Package

```
python
```

```
from utils.math_utils import add
```

```
print(add(10, 5))
```



8.7 Absolute vs Relative Imports (Conceptual)

Absolute Import (Preferred)

Absolute Import (Preferred)

python

```
from utils.math_utils import add
```

Relative Import (Advanced)

python

```
from .math_utils import add
```

👉 Teach **absolute first**, relative later.

8.8 Real-World Folder Structure (Professional)

CSS

app/

|

├─ main.py

├─ services/

| └─ calculator.py

| └─ auth.py

|

├─ models/

| └─ user.py

|

└─ utils/

└─ helpers.py



MINI PROJECT – Modular Student System

Goal

Split logic into multiple modules



Structure

CSS

student_app/

|

├─ student.py

├─ marks.py

└─ main.py

student.py

python

```
def create_student(name, age):  
    return {"name": name, "age": age}
```

marks.py

python

```
def calculate_total(marks):  
    return sum(marks)
```

main.py

python

```
from student import create_student  
from marks import calculate_total  
  
student = create_student("Aman", 21)  
marks = [80, 90, 85]  
  
total = calculate_total(marks)  
  
print(student)  
print("Total Marks:", total)
```

